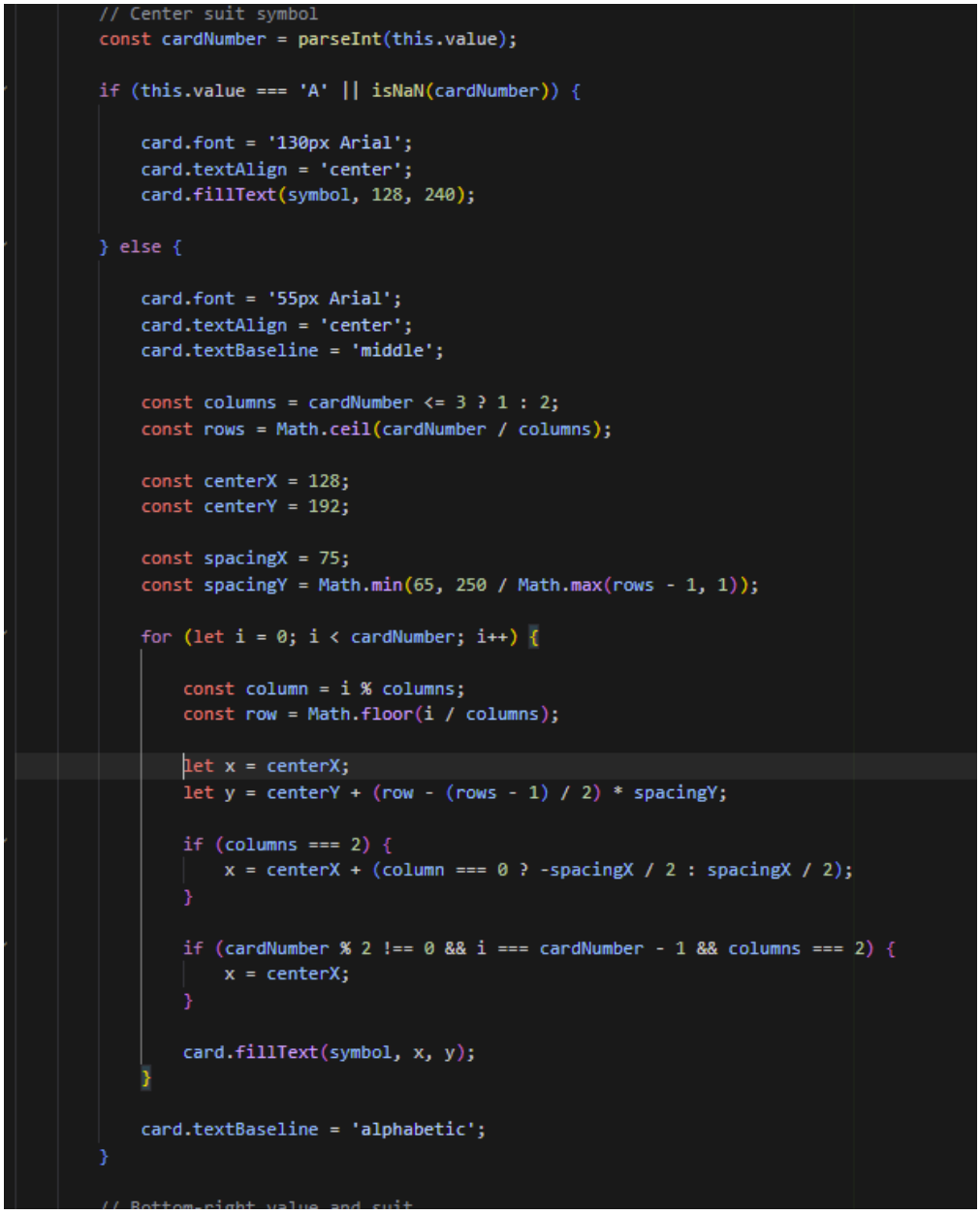
Works Cited Document - “We’re Going to War”
Jack P, Jack G, Abbi W

Jack Pelletier

When making the card deck,I ran into issues where the card had the proper number value attached to it, but every card just appeared as an Ace. So the Seven of Hearts would properly function as a Seven, but would only have a single heart on it. So I asked ChatGPT “How to make it so the cards have the matching numbers of suits as their number.” It helped me make this function where it would check the number and loop an equal numbers of times, creating one of the suit symbols on the front texture each time.



```
// Center suit symbol
const cardNumber = parseInt(this.value);

if (this.value === 'A' || isNaN(cardNumber)) {

    card.font = '130px Arial';
    card.textAlign = 'center';
    card.fillText(symbol, 128, 240);

} else {

    card.font = '55px Arial';
    card.textAlign = 'center';
    card.textBaseline = 'middle';

    const columns = cardNumber <= 3 ? 1 : 2;
    const rows = Math.ceil(cardNumber / columns);

    const centerX = 128;
    const centerY = 192;

    const spacingX = 75;
    const spacingY = Math.min(65, 250 / Math.max(rows - 1, 1));

    for (let i = 0; i < cardNumber; i++) {

        const column = i % columns;
        const row = Math.floor(i / columns);

        let x = centerX;
        let y = centerY + (row - (rows - 1) / 2) * spacingY;

        if (columns === 2) {
            x = centerX + (column === 0 ? -spacingX / 2 : spacingX / 2);
        }

        if (cardNumber % 2 !== 0 && i === cardNumber - 1 && columns === 2) {
            x = centerX;
        }

        card.fillText(symbol, x, y);
    }

    card.textBaseline = 'alphabetic';
}

// Bottom-right value and suit
```

I ran into issues when trying to make the face cards in the deck display properly. I brought in images to use for the Jack, Queen, and King cards, but when I tried to open the game through VSCode, I would get errors in the console saying that the browser's CORS policy was blocking access to the image files. I looked for solutions online and consulted with ChatGPT, which mentioned that encoding the images into Base64 would allow them to be loaded directly through the game's code instead of accessing the image files separately. To accomplish this, I tried using an online encoder, but when I tried to put the encoding into the file it would have errors about the grammar throughout the string. So I instead asked ChatGPT "how do I make an encoder for the images since the one online is not working" and it made the ConvertCards.html file, which allowed me to upload the card images and get the strings. I then uploaded those strings into a new FaceCardImages.js, and after making the proper calls with it, would finally allow the face cards to properly display on the cards.

```
<body>

    <h2>Convert Face Card Images</h2>

    <p>Select all 12 Jack, Queen, and King PNG images.</p>

    <input type="file" id="cardImages" accept="image/png" multiple>

    <button id="convertButton">Generate FaceCardImages.js</button>

    <p id="status"></p>

    <script>

        document.getElementById('convertButton').addEventListener('click', async () => {

            const files = Array.from(document.getElementById('cardImages').files);
            const status = document.getElementById('status');

            if (files.length !== 12) {
                status.textContent = 'Please select all 12 face-card images.';
                return;
            }

            const images = {};

            for (const file of files) {

                const dataURL = await new Promise((resolve, reject) => {

                    const reader = new FileReader();

                    reader.onload = () => resolve(reader.result);
                    reader.onerror = () => reject(reader.error);

                    reader.readAsDataURL(file);

                });

                const cardName = file.name.replace(/\.png$/i, '');

                images[cardName] = dataURL;
            }

            const content = 'const faceCardImages = ' +
                JSON.stringify(images, null, 4) + ';';

            const blob = new Blob([content], { type: 'text/javascript' });
            const url = URL.createObjectURL(blob);

            const link = document.createElement('a');
            link.href = url;
            link.download = 'FaceCardImages.js';
            link.click();

            URL.revokeObjectURL(url);

            status.textContent = 'FaceCardImages.js generated successfully!';
        });

    </script>

</body>
</html>
```

Jack Griffin

I used Google Gemini to help finalize the positions for the table and Bee model, given as I was trying to make it before the controls for moving it around were added, so it helped with getting an accurate look at how it would appear in the finished version of the project. I also needed help getting the gradients of the skybox to work correctly, and ensuring that the lighting features were working as intended. Some of my prompts included: “How can I make the wing cylinders of the bee model animated to emulate flying?” & “How can we make a skybox with gradient colors on the sides?”

Abbigail Weber

Tasked with handling the game logic, the biggest issue I struggled with was handling when there was a “ war” between players. I had a turn time both attempting to write the logic out as well as correctly animating the cards so that one was placed faced down and the other was placed faced up. After running into a lot of issues, I used chatGPT to try and correct the logic. Asking “help me understand why the war scenario is not functioning properly,” It showed me that I needed to add to the way cards were dealt into the pot, as well as correcting the logic of the war itself, as I had written it so only the players that tied were adding to the pot.

```
function beginWar() {
  const actions = [];
  faceUpCards = [];
  const eligiblePlayers = [];

  playerHands.forEach((hand, playerIndex) =>{

    if(hand.length === 0){
    return;
    }
 eligiblePlayers.push(playerIndex);

const position = playPostions[playerIndex];

 if(hand.length >= 2){
      const faceDownCard = hand.pop();
      playerCardCounts[playerIndex]--;
      cardPot.push(faceDownCard);


      actions.push({
        card: faceDownCard,

        target: new THREE.Vector3(
          position.x,
          cardBaseHeight + cardPot.length * 0.003,
          position.z
        ),
        faceUp: false
      });
    }

    const faceUpCard = hand.pop();
    playerCardCounts[playerIndex]--;
    cardPot.push(faceUpCard);

    faceUpCards.push({
      playerIndex: playerIndex,
      card: faceUpCard
    });
```

```
function dealCards(){

  if(dealtCount >= deck.length){
    return;
  }

  isDealing = true;

  const actions = [];
  const cardsToDeal = [];

  const pendingCards = [0, 0, 0];

  for(let i = dealtCount; i < deck.length; i++){
    const playerIndex = i % playerHands.length;

   const card = deck[deck.length - 1 - i];


  const player = players[playerIndex];

  const stackHeight = cardBaseHeight + (playerHands[playerIndex].length + pendingCards[playerIndex])* cardSpacing;

  const target = new THREE.Vector3(player.x, stackHeight, player.z);

    actions.push({
      card: card,
      target: target,
      faceUp: false
    });

    cardsToDeal.push({
      playerIndex: playerIndex,
      card: card
```